pointPredictions

January 29, 2026

1 All players from [SOMETHING]

Using the McDavid file I have made I'll try to build a database for players of a certain team or schedule. Basically a loop that loops through the player IDs a certain call to the API return.

1.1 Init

[1]:
```
# Straight from the book
import sys
assert sys.version_info >= (3, 5)

import pandas as pd

# Scikit-Learn  0.20 is required
import sklearn
assert sklearn.__version__ >= "0.20"

# Common imports
import numpy as np
import os

# To plot pretty figures
%matplotlib inline
import matplotlib as mpl
import matplotlib.pyplot as plt
mpl.rc('axes', labelsize=14)
mpl.rc('xtick', labelsize=12)
mpl.rc('ytick', labelsize=12)

# Where to save the figures
PROJECT_ROOT_DIR = "."
CHAPTER_ID = "personal_project"
IMAGES_PATH = os.path.join(PROJECT_ROOT_DIR, "images", CHAPTER_ID)
os.makedirs(IMAGES_PATH, exist_ok=True)

def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    path = os.path.join(IMAGES_PATH, fig_id + "." + fig_extension)
    print("Saving figure", fig_id)
```

```
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
from zlib import crc32

def test_set_check(identifier, test_ratio):
    return crc32(np.int64(identifier)) & 0xffffffff < test_ratio * 2**32

def split_train_test_by_id(data, test_ratio, id_column):
    ids = data[id_column]
    in_test_set = ids.apply(lambda id_: test_set_check(id_, test_ratio))
    return data.loc[~in_test_set], data.loc[in_test_set]
```

[2]:
```
# From the McDavid file
from nhlpy import NHLClient
import json

# Initialize the NHL client
client = NHLClient()

# Default storage path for player stats
STATS_PATH = os.path.join("datasets", "nhl_players")

def fetch_player_career_stats(player_id, stats_path=STATS_PATH):
    if not os.path.isdir(stats_path):
        os.makedirs(stats_path)

    file_path = os.path.join(stats_path, f"{player_id}_career_stats.json")

    # Check if file already exists
    if os.path.isfile(file_path):
        with open(file_path, "r") as f:
            return json.load(f)

    # Fetch career stats from NHL API
    career_stats = client.stats.player_career_stats(player_id=player_id)

    # Save JSON locally
    with open(file_path, "w") as f:
        json.dump(career_stats, f, indent=2)

    return career_stats
```

[3]:
```
# These are the client calls :
# games = client.schedule.daily_schedule(), maybe to implement later
# roster = client.teams.team_roster(team_abbr="BUF", season="20242025")
```

```
def fetch_roster(team_abbr, season, stats_path=STATS_PATH):
    """
    Fetches a player ID list from NHL API and returns an array of integers.

    Args:
        team_abbr (str): NHL team abbreviation "XXX"
        season (str): Season id "YYYYYYYY"
        stats_path (str): Path to save the JSON file
    Returns:
        filepath
    """
    if not os.path.isdir(stats_path):
        os.makedirs(stats_path/{season})

    file_path = os.path.join(stats_path, f"{team_abbr}_roster.json")

    # No check for existing file because if current season we might need to␣
 ↪update it anyway

    roster = client.teams.team_roster(team_abbr, season)

    with open(file_path, "w") as f:
        json.dump(roster, f, indent=2)

    return file_path
```

[4]:
```
# Cleaning up the roster to keep only the player IDs
import json
import pandas as pd

def query_player_ids(file_path):
    with open(file_path, encoding='utf-8-sig') as f:
        data = json.load(f)
    # Suppose the career stats are under 'stats' key
    dff = pd.json_normalize(data['forwards'])
    dfd = pd.json_normalize(data['defensemen'])

    ids = pd.concat([dff['id'], dfd['id']], ignore_index=True)
    return ids
```

[5]:
```
# This is if I want a specific team
# ids = query_player_ids(fetch_roster("MTL", "20252026"))

# This gives all the teams
teams = client.teams.teams()

# So we write an abbreviation list
```

```
def query_team_abbreviations():
    teams = client.teams.teams()
    team_abbreviations = pd.json_normalize(teams)
    abbreviations = team_abbreviations['abbr']#.reset_index(drop=True)
    return abbreviations
```

[6]:
```
def store_team_stats(team_ids):
    for id in team_ids:
        fetch_player_career_stats(id)
        # Source - https://stackoverflow.com/a
        # Posted by Martin Evans
        # Retrieved 2026-01-08, License - CC BY-SA 4.0
        # And chatGPT for the logic to clean the data

        with open(f'./datasets/nhl_players/{id}_career_stats.json',␣
 ↪encoding='utf-8-sig') as f:
            data = json.load(f)

        # Suppose the career stats are under 'stats' key
        df = pd.json_normalize(data['seasonTotals'])

        # Save to CSV
        df.to_csv('season_totals.csv', index=False, encoding='utf-8')

        # Keep only rows where leagueAbbrev is "NHL" and gameTypeID is 2
        df = df[(df['leagueAbbrev'] == 'NHL') & (df['gameTypeId'] == 2)]
        df = df.dropna(axis=1, how='all')

        # Save the filtered CSV
        df.to_csv(f"./datasets/nhl_players/{id}_season_totals.csv", index=False)
```

[25]:
```
ids = []
def store_all_active_players():
    abbrs = query_team_abbreviations()
    for team_abbr in abbrs:
        file_path = fetch_roster(team_abbr, 20252026)
        idss = (query_player_ids(file_path))
        ids.append(idss)
        store_team_stats(idss)
    return ids
```

[26]:
```
store_all_active_players()
```

[26]:
```
[0     8482947
 1     8484149
 2     8479525
 3     8480835
```

```
4     8480448
5     8481641
6     8476455
7     8477476
8     8477492
9     8482953
10    8480039
11    8475754
12    8477501
13    8478109
14    8470613
15    8479398
16    8480069
17    8484258
18    8476312
19    8479387
20    8478038
Name: id, dtype: int64,
0     8477416
1     8478519
2     8480876
3     8482090
4     8476878
5     8482201
6     8476826
7     8477404
8     8479542
9     8480995
10    8483752
11    8476453
12    8477426
13    8478010
14    8483398
15    8478416
16    8481719
17    8480426
18    8482168
19    8475167
20    8482929
21    8474151
22    8482655
23    8478178
Name: id, dtype: int64,
0     8481557
1     8478493
2     8475220
3     8477451
```

```
4     8476994
5     8475149
6     8478864
7     8475752
8     8481477
9     8475765
10    8478508
11    8483525
12    8475692
13    8474567
14    8476463
15    8482122
16    8480800
17    8482094
18    8483460
19    8482641
20    8478136
21    8474716
Name: id, dtype: int64,
0     8478427
1     8482809
2     8477478
3     8477940
4     8475791
5     8476873
6     8482093
7     8480829
8     8476921
9     8481491
10    8480762
11    8473533
12    8482702
13    8480830
14    8478970
15    8476906
16    8484428
17    8480817
18    8482100
19    8482911
20    8476422
21    8476958
22    8480336
Name: id, dtype: int64,
0     8479414
1     8473994
2     8476278
3     8482145
```

```
4     8480840
5     8475168
6     8477454
7     8476889
8     8478449
9     8484829
10    8482740
11    8478420
12    8480027
13    8475794
14    8479351
15    8483425
16    8478476
17    8481581
18    8480036
19    8476902
20    8480878
21    8480950
22    8475755
Name: id, dtype: int64,
0     8478891
1     8477456
2     8477429
3     8479337
4     8484471
5     8474141
6     8483464
7     8477946
8     8479992
9     8482078
10    8481725
11    8474037
12    8480879
13    8475279
14    8482762
15    8476979
16    8474612
17    8481607
18    8484223
19    8481542
Name: id, dtype: int64,
0     8484145
1     8479941
2     8482659
3     8482623
4     8478413
5     8484797
```

```
6     8482896
7     8481522
8     8483468
9     8479359
10    8480802
11    8480064
12    8483500
13    8482097
14    8479420
15    8477949
16    8475722
17    8480196
18    8481524
19    8480839
20    8481708
21    8480891
22    8484305
23    8482671
24    8480807
25    8479982
Name: id, dtype: int64,
0     8476981
1     8478104
2     8482737
3     8481540
4     8481523
5     8476479
6     8484984
7     8478133
8     8475848
9     8482775
10    8479339
11    8481618
12    8483515
13    8480018
14    8480074
15    8480813
16    8478851
17    8480865
18    8482087
19    8483457
20    8476875
21    8481593
22    8482964
Name: id, dtype: int64,
0     8478042
1     8479591
```

```
2     8479987
3     8479661
4     8480355
5     8482177
6     8476374
7     8477496
8     8483489
9     8479999
10    8477956
11    8483505
12    8482634
13    8478401
14    8481219
15    8480887
16    8480035
17    8476854
18    8482511
19    8479325
20    8479369
21    8477507
Name: id, dtype: int64,
0     8478569
1     8479638
2     8482475
3     8471675
4     8480980
5     8480842
6     8475763
7     8485414
8     8481481
9     8471215
10    8477511
11    8483487
12    8478438
13    8476483
14    8475810
15    8477365
16    8477435
17    8474578
18    8476967
19    8471724
20    8478854
21    8482470
22    8481030
23    8478450
Name: id, dtype: int64,
0     8477964
```

```
1     8483890
2     8481604
3     8478403
4     8476881
5     8482125
6     8479353
7     8476448
8     8478434
9     8479550
10    8478483
11    8481133
12    8476438
13    8476925
14    8475191
15    8475913
16    8478397
17    8478396
18    8477018
19    8481527
20    8478468
21    8475188
22    8477447
Name: id, dtype: int64,
0     8478445
1     8475231
2     8477494
3     8477407
4     8483553
5     8482476
6     8481601
7     8477500
8     8475314
9     8481237
10    8476419
11    8476292
12    8475151
13    8484221
14    8485702
15    8480871
16    8477950
17    8476429
18    8476917
19    8477506
20    8481014
21    8485366
22    8477369
Name: id, dtype: int64,
```

```
0     8477934
1     8479365
2     8474641
3     8475786
4     8477406
5     8477953
6     8477508
7     8478233
8     8478402
9     8476454
10    8481617
11    8478458
12    8484509
13    8483512
14    8480803
15    8475218
16    8480834
17    8477498
18    8480831
19    8481056
20    8478013
Name: id, dtype: int64,
0     8484388
1     8479619
2     8483431
3     8478474
4     8482699
5     8480849
6     8479343
7     8477021
8     8480855
9     8477070
10    8482175
11    8477951
12    8478831
13    8479293
14    8479977
15    8474013
16    8480084
17    8480434
18    8478507
19    8476874
20    8477220
21    8479410
22    8484386
Name: id, dtype: int64,
0     8477493
```

```
1     8477935
2     8480003
3     8479981
4     8478421
5     8479316
6     8482113
7     8480185
8     8473419
9     8477933
10    8478542
11    8482713
12    8481655
13    8479314
14    8477409
15    8483771
16    8484304
17    8477932
18    8478055
19    8477495
20    8475179
21    8478859
22    8473507
23    8482131
Name: id, dtype: int64,
0     8484153
1     8482118
2     8483445
3     8475798
4     8478424
5     8477527
6     8473986
7     8475184
8     8482745
9     8481754
10    8480068
11    8484762
12    8476458
13    8478873
14    8478366
15    8479705
16    8485512
17    8475462
18    8481563
19    8481605
20    8483490
21    8482178
22    8476885
```

```
23    8482803
Name: id, dtype: int64,
0     8484801
1     8480848
2     8482667
3     8478874
4     8476624
5     8484911
6     8480798
7     8485402
8     8482859
9     8471817
10    8483630
11    8480748
12    8475784
13    8484227
14    8475726
15    8477505
16    8479576
17    8484806
18    8479983
19    8482861
20    8475906
21    8480043
22    8482166
23    8475200
Name: id, dtype: int64,
0     8476469
1     8482124
2     8483675
3     8477942
4     8477998
5     8482726
6     8477960
7     8471685
8     8483808
9     8482155
10    8481732
11    8482408
12    8479675
13    8470621
14    8481532
15    8483406
16    8479998
17    8476879
18    8482730
19    8474563
```

```
20    8475208
21    8476441
22    8479421
Name: id, dtype: int64,
0     8482665
1     8484800
2     8474586
3     8477919
4     8481554
5     8481789
6     8477955
7     8482874
8     8483570
9     8475768
10    8476905
11    8480009
12    8482751
13    8483524
14    8478407
15    8482858
16    8479985
17    8476457
18    8479324
19    8479372
20    8477986
21    8476467
Name: id, dtype: int64,
0     8479022
1     8484142
2     8481553
3     8480220
4     8476461
5     8475235
6     8477989
7     8482159
8     8483733
9     8479336
10    8477903
11    8478439
12    8484387
13    8478967
14    8480015
15    8481533
16    8482126
17    8482142
18    8478454
19    8477499
```

```
20    8477948
21    8476372
22    8481546
Name: id, dtype: int64,
0     8475745
1     8484166
2     8478046
3     8476432
4     8482660
5     8480806
6     8480893
7     8478975
8     8477497
9     8479671
10    8482705
11    8481716
12    8477425
13    8481161
14    8479371
15    8475790
16    8483485
17    8478500
18    8476923
19    8474090
20    8478460
21    8481178
Name: id, dtype: int64,
0     8478020
1     8480208
2     8476393
3     8481528
4     8474189
5     8473512
6     8482092
7     8483676
8     8477073
9     8474102
10    8481596
11    8482116
12    8480801
13    8480188
14    8478469
15    8475324
16    8482095
17    8484321
18    8482105
19    8481606
```

```
20    8482245
Name: id, dtype: int64,
0     8484158
1     8477503
2     8475714
3     8478057
4     8482720
5     8476872
6     8478904
7     8481711
8     8479318
9     8482259
10    8477939
11    8484901
12    8481582
13    8478462
14    8475166
15    8481122
16    8478443
17    8475171
18    8476931
19    8479026
20    8476853
21    8483546
22    8479442
23    8475690
Name: id, dtype: int64,
0     8478463
1     8475343
2     8479400
3     8479520
4     8483573
5     8482148
6     8484186
7     8481580
8     8477947
9     8471214
10    8481656
11    8482088
12    8478440
13    8476880
14    8474590
15    8480990
16    8479345
17    8480796
18    8475795
19    8478911
```

```
20    8480873
21    8477845
Name: id, dtype: int64,
0     8479407
1     8477015
2     8481032
3     8474149
4     8479996
5     8476822
6     8481721
7     8484177
8     8480002
9     8481559
10    8477996
11    8479772
12    8483397
13    8478414
14    8482110
15    8476474
16    8484958
17    8475455
18    8476462
19    8482684
20    8480192
21    8483495
22    8477488
23    8478399
24    8478841
Name: id, dtype: int64,
0     8478047
1     8482146
2     8476887
3     8475287
4     8479370
5     8476539
6     8477446
7     8475158
8     8482062
9     8474564
10    8482103
11    8484241
12    8482111
13    8483565
14    8477971
15    8479980
16    8474600
17    8480246
```

```
18    8476869
19    8482482
Name: id, dtype: int64,
0     8484144
1     8477479
2     8477444
3     8482703
4     8477450
5     8477987
6     8473422
7     8483450
8     8478043
9     8484185
10    8481624
11    8484197
12    8483493
13    8482172
14    8476882
15    8481806
16    8476891
17    8482176
18    8484783
19    8476473
20    8481568
Name: id, dtype: int64,
0     8477380
1     8475842
2     8482157
3     8481726
4     8483690
5     8482109
6     8476468
7     8482747
8     8478550
9     8484210
10    8479390
11    8482460
12    8477839
13    8476389
14    8476459
15    8478840
16    8479323
17    8478882
18    8482067
19    8482666
20    8481525
21    8482073
```

```
22    8480001
Name: id, dtype: int64,
0     8480289
1     8478398
2     8480113
3     8481043
4     8476392
5     8476480
6     8475799
7     8474679
8     8476871
9     8482149
10    8476460
11    8473604
12    8480014
13    8476331
14    8477938
15    8476525
16    8477504
17    8482192
18    8480145
19    8483510
20    8480049
21    8474568
22    8479378
Name: id, dtype: int64,
0     8474150
1     8481556
2     8476399
3     8482679
4     8480797
5     8480028
6     8484860
7     8484180
8     8476456
9     8475172
10    8477993
11    8483609
12    8479066
13    8481028
14    8481068
15    8482074
16    8480860
17    8479402
18    8484150
19    8477810
20    8482165
```

```
21    8481167
22    8484768
23    8477346
24    8480727
Name: id, dtype: int64,
0     8481013
1     8475760
2     8477402
3     8484164
4     8477952
5     8482077
6     8478472
7     8479385
8     8479644
9     8481070
10    8482089
11    8475170
12    8483516
13    8476897
14    8480459
15    8480023
16    8480281
17    8477573
18    8481598
19    8475753
20    8475764
21    8482516
22    8482733
23    8476892
24    8481006
Name: id, dtype: int64,
0     8482496
1     8476927
2     8478444
3     8480078
4     8478498
5     8478856
6     8481535
7     8480144
8     8475169
9     8481024
10    8483476
11    8482055
12    8483499
13    8480012
14    8482691
15    8482079
```

```
16    8484136
17    8484798
18    8475762
19    8479425
20    8480058
21    8483768
22    8474574
23    8483678
24    8477969
25    8484240
Name: id, dtype: int64]
```

[27]:
```
id_list = pd.DataFrame(ids)
id_list = id_list.values.flatten().tolist()

id_list = [str(int(id)) for id in id_list if pd.notna(id)]
```

[28]:
```
def load_hockey_data(filename):
    try:
        return pd.read_csv(f'./datasets/nhl_players/{filename}_season_totals.
 ↪csv')
    except pd.errors.EmptyDataError:
        print(f"No data found for {filename}.")
        return pd.DataFrame()
```

[29]:
```
hockey_data_list = []

for id in id_list:
    hockey_data = load_hockey_data(id)
    hockey_data_list.append({'id': id, 'data': hockey_data})


hockey_df = pd.DataFrame(hockey_data_list)
```

```
No data found for 8483675.
```

[30]:
```
import pandas as pd

# Initialize an empty list for cleaned data
cleaned_data_list = []

for id in id_list:
    hockey_data = load_hockey_data(id)  # Load the data

    # Check if hockey_data is a DataFrame
    if isinstance(hockey_data, pd.DataFrame):
        hockey_data['id'] = id  # Add the ID to the DataFrame
```

```
        cleaned_data_list.append(hockey_data)  # Append the DataFrame to the␣
↪list
    else:
        print(f"Data for ID {id} is not a DataFrame: {hockey_data}")

# Concatenate all DataFrames into one
hockey_df_cleaned = pd.concat(cleaned_data_list, ignore_index=True)
```

```
No data found for 8483675.
```

[31]:
```
hockey_df_cleaned.describe()
hockey_df_cleaned.head()
```

[31]:

	assists	gameTypeId	gamesPlayed	goals	leagueAbbrev	pim	points	\
0	6.0	2.0	30.0	1.0	NHL	6.0	7.0	
1	0.0	2.0	1.0	0.0	NHL	0.0	0.0	
2	4.0	2.0	30.0	5.0	NHL	8.0	9.0	
3	3.0	2.0	30.0	9.0	NHL	16.0	12.0	
4	17.0	2.0	79.0	22.0	NHL	24.0	39.0	

	season	sequence	teamName.default	…	shootingPctg	\
0	20252026.0	1.0	Colorado Avalanche	…	0.052632	
1	20232024.0	1.0	Columbus Blue Jackets	…	0.000000	
2	20252026.0	1.0	Colorado Avalanche	…	0.138889	
3	20202021.0	1.0	Tampa Bay Lightning	…	0.196000	
4	20212022.0	1.0	Tampa Bay Lightning	…	0.138000	

	shorthandedGoals	shorthandedPoints	shots	teamCommonName.default	\
0	0.0	0.0	19.0	Avalanche	
1	0.0	0.0	0.0	Blue Jackets	
2	0.0	0.0	36.0	Avalanche	
3	0.0	0.0	46.0	Lightning	
4	0.0	0.0	159.0	Lightning	

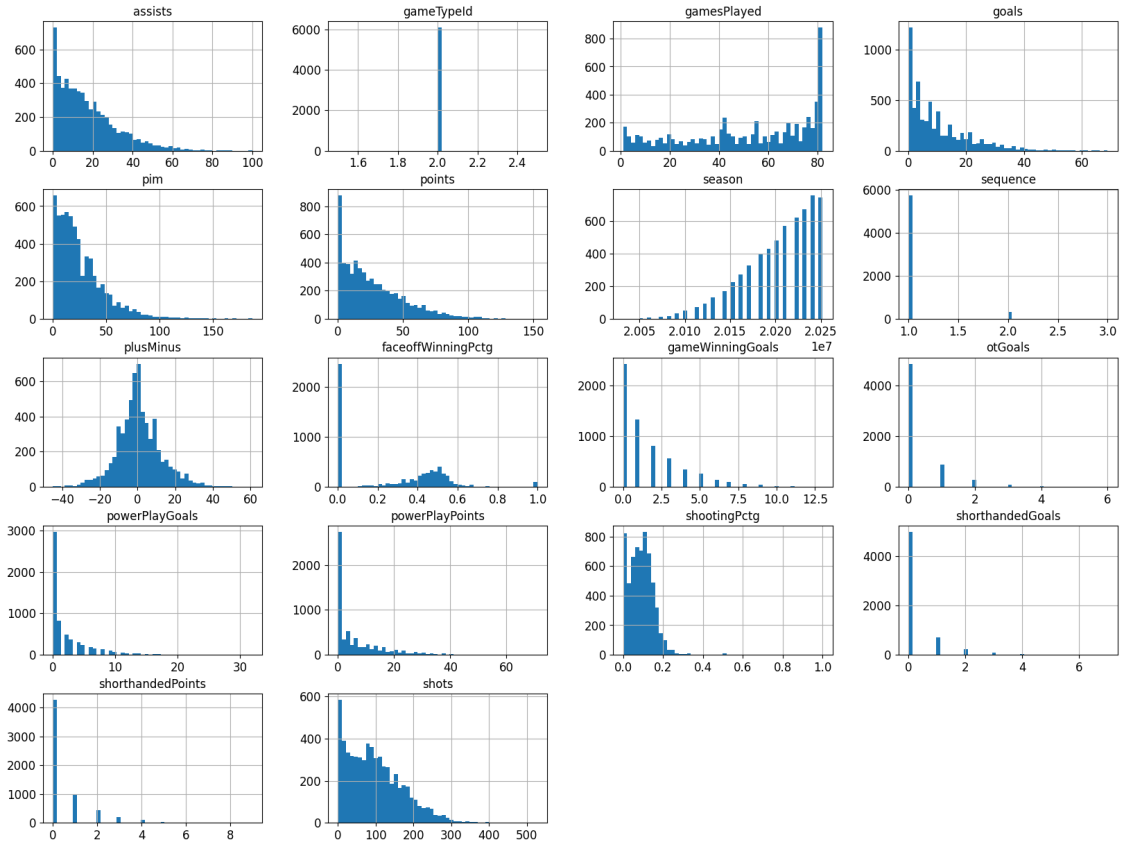
	teamName.fr	teamPlaceNameWithPreposition.default	\
0	Avalanche du Colorado	Colorado	
1	Blue Jackets de Columbus	Columbus	
2	Avalanche du Colorado	Colorado	
3	Lightning de Tampa Bay	Tampa Bay	
4	Lightning de Tampa Bay	Tampa Bay	

	teamPlaceNameWithPreposition.fr	id	teamCommonName.fr
0	du Colorado	8482947	NaN
1	de Columbus	8484149	NaN
2	du Colorado	8484149	NaN
3	de Tampa Bay	8479525	NaN
4	de Tampa Bay	8479525	NaN

[5 rows x 27 columns]

[32]:
```
import matplotlib.pyplot as plt

hockey_df_cleaned.hist(bins=50, figsize=(20, 15))
plt.show()
```



[33]:
```
hockey = hockey_df_cleaned
```

[34]:
```
# to make this notebook's output identical at every run
np.random.seed(42)
```

[35]:
```
from sklearn.model_selection import train_test_split
train_set, test_set = train_test_split(hockey, test_size=0.2, random_state=42)
```

[36]:
```
train_set, test_set = train_test_split(hockey, test_size=0.2, random_state=42)
```

[37]:
```
test_set.head()
```

[37]:
```
      assists  gameTypeId  gamesPlayed  goals leagueAbbrev   pim  points  \
1046     18.0         2.0         45.0   22.0          NHL  32.0    40.0
4205     14.0         2.0         63.0   18.0          NHL  60.0    32.0
4601     56.0         2.0         82.0   27.0          NHL  12.0    83.0
2410     12.0         2.0         79.0   14.0          NHL  26.0    26.0
167      22.0         2.0         68.0    6.0          NHL  16.0    28.0

          season  sequence    teamName.default  …  shootingPctg  \
1046  20252026.0       1.0   Detroit Red Wings  …      0.169231
4205  20232024.0       1.0   Vancouver Canucks  …      0.214286
4601  20232024.0       1.0   New Jersey Devils  …      0.108871
2410  20212022.0       1.0   Vancouver Canucks  …      0.122000
167   20192020.0       1.0  New York Islanders  …      0.049000

     shorthandedGoals  shorthandedPoints  shots  teamCommonName.default  \
1046              0.0                0.0  130.0               Red Wings
4205              0.0                0.0   84.0                 Canucks
4601              0.0                1.0  248.0                  Devils
2410              0.0                0.0  115.0                 Canucks
167               0.0                0.0  122.0               Islanders

               teamName.fr  teamPlaceNameWithPreposition.default  \
1046   Red Wings de Detroit                              Detroit
4205   Canucks de Vancouver                            Vancouver
4601   Devils du New Jersey                           New Jersey
2410   Canucks de Vancouver                            Vancouver
167   Islanders de New York                             New York

      teamPlaceNameWithPreposition.fr       id  teamCommonName.fr
1046                       de Detroit  8477946                NaN
4205                     de Vancouver  8478057                NaN
4601                    du New Jersey  8479407                NaN
2410                     de Vancouver  8481617                NaN
167                       de New York  8478038                NaN

[5 rows x 27 columns]
```

[38]:
```
hockey["points"].describe()
```

[38]:
```
count    6082.000000
mean       26.984709
std        23.395148
min         0.000000
25%         8.000000
50%        21.000000
75%        40.000000
max       153.000000
```

```
Name: points, dtype: float64
```

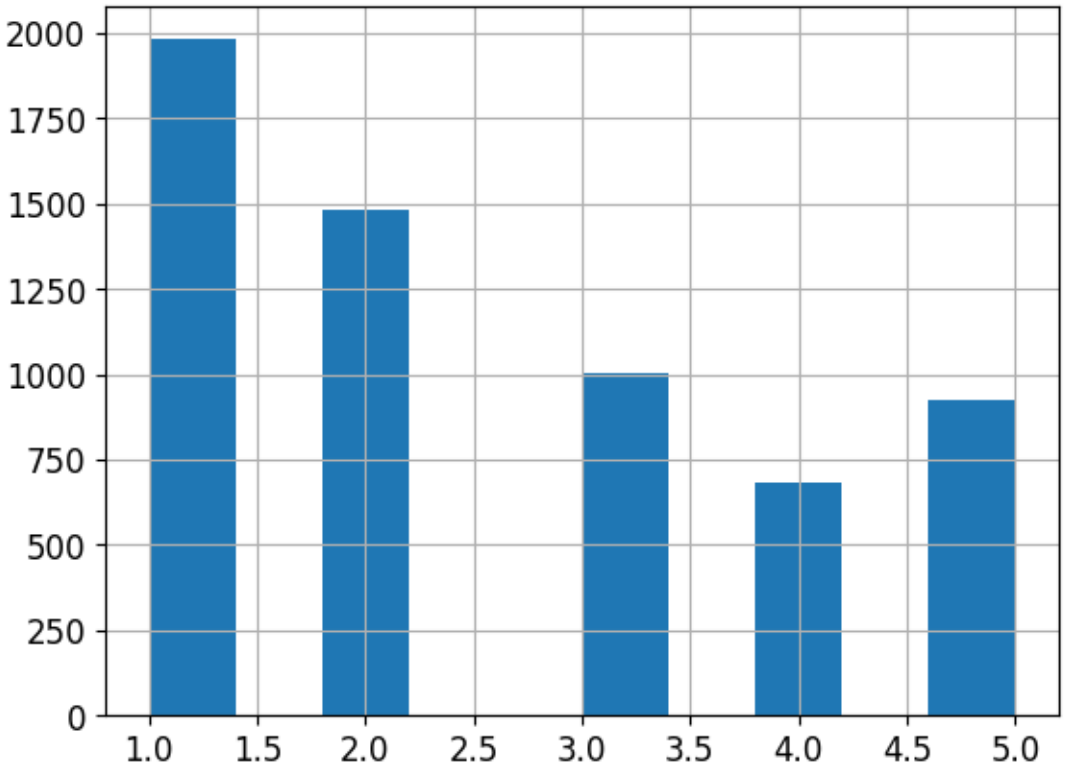
[39]:
```
hockey["points_cat"] = pd.cut(
    hockey["points"],
    bins=[-1, 12, 25, 38, 51, np.inf],
    labels=[1, 2, 3, 4, 5])
```

[40]:
```
hockey["points_cat"].value_counts()
```

[40]:
```
points_cat
1    1982
2    1485
3    1006
5     927
4     682
Name: count, dtype: int64
```

[41]:
```
hockey["points_cat"].hist()
```

[41]:
```
<Axes: >
```



[42]:
```
nan_rows = hockey[hockey["points_cat"].isna()]
print(nan_rows)
hockey_no_nan = hockey.dropna(subset=["points_cat"])
hockey_no_nan
```

```
Empty DataFrame
Columns: [assists, gameTypeId, gamesPlayed, goals, leagueAbbrev, pim, points,
season, sequence, teamName.default, plusMinus, avgToi, faceoffWinningPctg,
gameWinningGoals, otGoals, powerPlayGoals, powerPlayPoints, shootingPctg,
shorthandedGoals, shorthandedPoints, shots, teamCommonName.default, teamName.fr,
teamPlaceNameWithPreposition.default, teamPlaceNameWithPreposition.fr, id,
teamCommonName.fr, points_cat]
Index: []

[0 rows x 28 columns]
```

[42]:

	assists	gameTypeId	gamesPlayed	goals	leagueAbbrev	pim	points	\
0	6.0	2.0	30.0	1.0	NHL	6.0	7.0	
1	0.0	2.0	1.0	0.0	NHL	0.0	0.0	
2	4.0	2.0	30.0	5.0	NHL	8.0	9.0	
3	3.0	2.0	30.0	9.0	NHL	16.0	12.0	
4	17.0	2.0	79.0	22.0	NHL	24.0	39.0	
…	…	…	…	…	…	…	…	
6077	26.0	2.0	82.0	4.0	NHL	44.0	30.0	
6078	15.0	2.0	47.0	3.0	NHL	25.0	18.0	
6079	10.0	2.0	31.0	1.0	NHL	20.0	11.0	
6080	8.0	2.0	43.0	1.0	NHL	45.0	9.0	
6081	10.0	2.0	33.0	2.0	NHL	8.0	12.0	

	season	sequence	teamName.default	…	shorthandedGoals	\
0	20252026.0	1.0	Colorado Avalanche	…	0.0	
1	20232024.0	1.0	Columbus Blue Jackets	…	0.0	
2	20252026.0	1.0	Colorado Avalanche	…	0.0	
3	20202021.0	1.0	Tampa Bay Lightning	…	0.0	
4	20212022.0	1.0	Tampa Bay Lightning	…	0.0	
…	…	…	…	…	…	
6077	20232024.0	1.0	Pittsburgh Penguins	…	0.0	
6078	20242025.0	1.0	Pittsburgh Penguins	…	0.0	
6079	20242025.0	2.0	Vancouver Canucks	…	0.0	
6080	20252026.0	1.0	Vancouver Canucks	…	0.0	
6081	20252026.0	1.0	Vancouver Canucks	…	0.0	

	shorthandedPoints	shots	teamCommonName.default	\
0	0.0	19.0	Avalanche	
1	0.0	0.0	Blue Jackets	
2	0.0	36.0	Avalanche	
3	0.0	46.0	Lightning	
4	0.0	159.0	Lightning	

```
…                 …    …                    …
6077              0.0  106.0                Penguins
6078              1.0   43.0                Penguins
6079              0.0   27.0                 Canucks
6080              0.0   35.0                 Canucks
6081              0.0   29.0                 Canucks

                  teamName.fr  teamPlaceNameWithPreposition.default  \
0        Avalanche du Colorado                              Colorado
1     Blue Jackets de Columbus                              Columbus
2        Avalanche du Colorado                              Colorado
3       Lightning de Tampa Bay                             Tampa Bay
4       Lightning de Tampa Bay                             Tampa Bay
…                          …                                    …
6077    Penguins de Pittsburgh                            Pittsburgh
6078    Penguins de Pittsburgh                            Pittsburgh
6079      Canucks de Vancouver                             Vancouver
6080      Canucks de Vancouver                             Vancouver
6081      Canucks de Vancouver                             Vancouver

      teamPlaceNameWithPreposition.fr       id  teamCommonName.fr  points_cat
0                        du Colorado  8482947                NaN           1
1                        de Columbus  8484149                NaN           1
2                        du Colorado  8484149                NaN           1
3                       de Tampa Bay  8479525                NaN           1
4                       de Tampa Bay  8479525                NaN           4
…                                …        …                  …         …
6077                   de Pittsburgh  8477969                NaN           3
6078                   de Pittsburgh  8477969                NaN           2
6079                    de Vancouver  8477969                NaN           1
6080                    de Vancouver  8477969                NaN           1
6081                    de Vancouver  8484240                NaN           1

[6082 rows x 28 columns]
```

[43]:
```
from sklearn.model_selection import StratifiedShuffleSplit

split = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
for train_index, test_index in split.split(hockey, hockey["points_cat"]):
    strat_train_set = hockey.loc[train_index]
    strat_test_set = hockey.loc[test_index]
```

[44]:
```
strat_test_set["points_cat"].value_counts() / len(strat_test_set)
```

[44]:
```
points_cat
1    0.326212
2    0.244043
```

```
3    0.165160
5    0.152835
4    0.111750
Name: count, dtype: float64
```

[45]:
```
hockey["points_cat"].value_counts() / len(hockey)
```

[45]:
```
points_cat
1    0.325880
2    0.244163
3    0.165406
5    0.152417
4    0.112134
Name: count, dtype: float64
```

[46]:
```
def points_cat_proportions(data):
    return data["points_cat"].value_counts() / len(data)

train_set, test_set = train_test_split(hockey, test_size=0.2, random_state=42)

compare_props = pd.DataFrame({
    "Overall": points_cat_proportions(hockey),
    "Stratified": points_cat_proportions(strat_test_set),
    "Random": points_cat_proportions(test_set),
}).sort_index()
compare_props["Rand. %error"] = 100 * compare_props["Random"] /␣
 ↪compare_props["Overall"] - 100
compare_props["Strat. %error"] = 100 * compare_props["Stratified"] /␣
 ↪compare_props["Overall"] - 100
```

[47]:
```
compare_props
```

[47]:

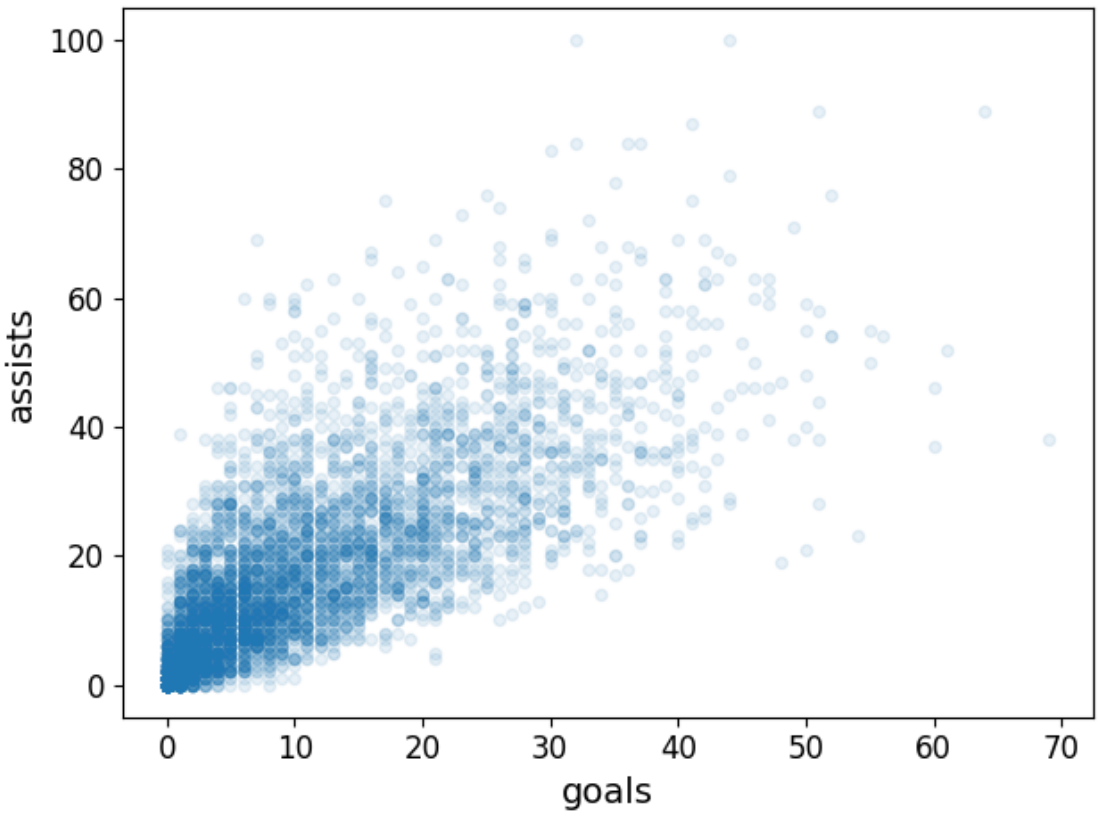
points_cat	Overall	Stratified	Random	Rand. %error	Strat. %error
1	0.325880	0.326212	0.327855	0.606278	0.101986
2	0.244163	0.244043	0.244043	-0.049302	-0.049302
3	0.165406	0.165160	0.175842	6.309391	-0.148656
4	0.112134	0.111750	0.098603	-12.066834	-0.342412
5	0.152417	0.152835	0.153657	0.813272	0.274163

1.2 Discover and Visualize the Data to Gain Insights

[48]:
```
hockey = strat_train_set.copy()
```

[49]:
```
hockey.plot(kind="scatter", x="goals", y="assists", alpha =0.1)
save_fig("bad_visualization_plot")
```

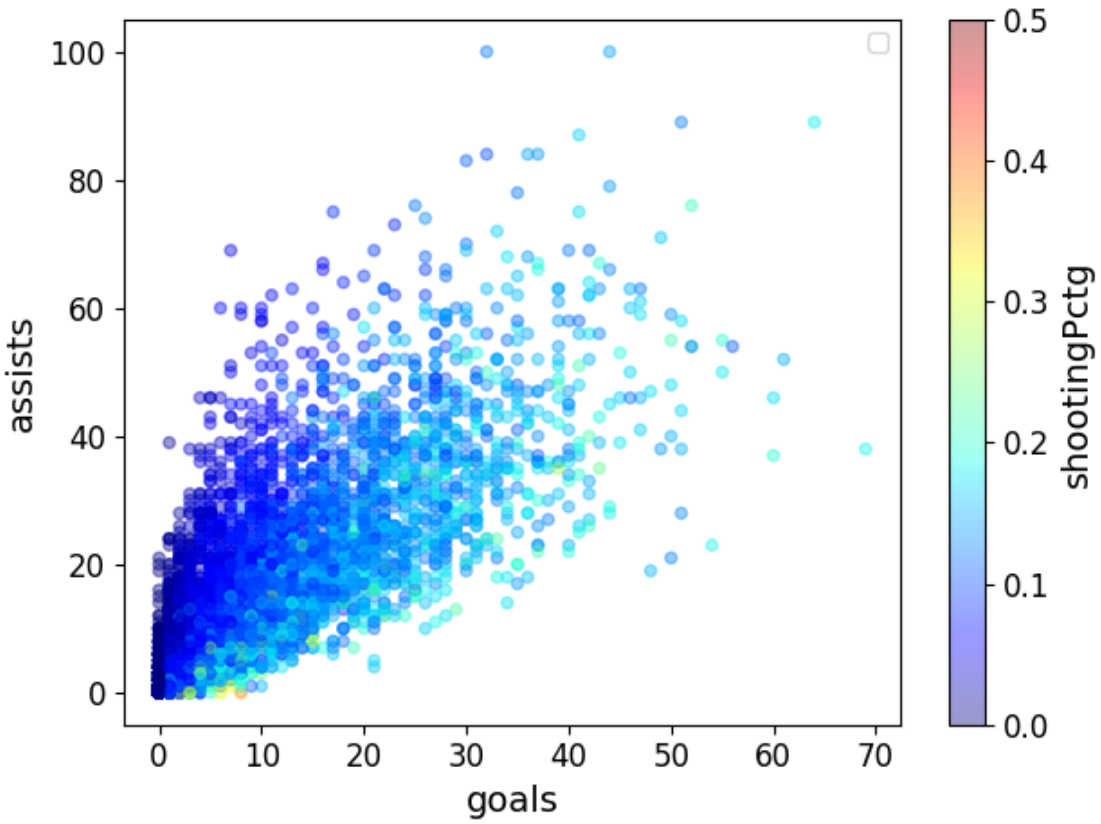
```
Saving figure bad_visualization_plot
```



[50]:
```
hockey.plot(kind="scatter", x="goals", y="assists", alpha=0.4,
            c="shootingPctg", cmap=plt.get_cmap("jet"), colorbar=True, vmax =␣
 ↪0.5,
            sharex=False)
plt.legend()
save_fig("hockey_prices_scatterplot")
```

```
/tmp/ipykernel_97390/200831888.py:4: UserWarning: No artists with labels found
to put in legend.  Note that artists whose label start with an underscore are
ignored when legend() is called with no argument.
  plt.legend()
```

```
Saving figure hockey_prices_scatterplot
```



1.2.1 Correlations

[51]:
```
# Only numeric columns
numeric_hockey = hockey.select_dtypes(include='number')
corr_matrix = numeric_hockey.corr()
corr_matrix["points"].sort_values(ascending=False)
```

[51]:
```
points                1.000000
assists               0.957812
goals                 0.912502
shots                 0.907503
powerPlayPoints       0.894850
powerPlayGoals        0.796501
gameWinningGoals      0.763621
gamesPlayed           0.699057
otGoals               0.522420
shootingPctg          0.435471
pim                   0.350648
shorthandedPoints     0.306860
shorthandedGoals      0.269857
```

```
faceoffWinningPctg    0.261611
plusMinus             0.247645
season               -0.138612
sequence             -0.185686
gameTypeId                 NaN
Name: points, dtype: float64
```

[52]:
```
hockey["points_per_pppoints"] = hockey["points"]/hockey["powerPlayPoints"]
hockey["points_per_game"] = hockey["points"]/hockey["gamesPlayed"]
hockey["points_per_plusMinus"] = hockey["points"]/(hockey["plusMinus"])
```

[53]:
```
# Only numeric columns
numeric_hockey = hockey.select_dtypes(include='number')
corr_matrix = numeric_hockey.corr()
corr_matrix["points"].sort_values(ascending=False)
```

[53]:
```
points                 1.000000
assists                0.957812
goals                  0.912502
shots                  0.907503
powerPlayPoints        0.894850
points_per_game        0.862058
powerPlayGoals         0.796501
gameWinningGoals       0.763621
gamesPlayed            0.699057
otGoals                0.522420
shootingPctg           0.435471
pim                    0.350648
shorthandedPoints      0.306860
shorthandedGoals       0.269857
faceoffWinningPctg     0.261611
plusMinus              0.247645
points_per_plusMinus   0.122346
season                -0.138612
sequence              -0.185686
points_per_pppoints   -0.232467
gameTypeId                  NaN
Name: points, dtype: float64
```

1.3 Prepare data

[54]:
```
hockey = strat_train_set.drop("points", axis=1) # drop labels for training set
# Dropping the number of assists
hockey = hockey.drop("assists", axis=1)
hockey = hockey.drop("goals", axis=1)

hockey_labels = strat_train_set["points"].copy()
```

```
numeric_hockey = hockey.select_dtypes(include='number')
```

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy="median")
```

```
imputer.fit(numeric_hockey)
imputer.statistics_
```

[54]:
```
array([2.0000000e+00, 5.6000000e+01, 2.0000000e+01, 2.0212022e+07,
       1.0000000e+00, 0.0000000e+00, 3.3333300e-01, 1.0000000e+00,
       0.0000000e+00, 1.0000000e+00, 2.0000000e+00, 8.8889000e-02,
       0.0000000e+00, 0.0000000e+00, 8.8000000e+01])
```

[55]:
```
numeric_hockey.median().values
```

[55]:
```
array([2.0000000e+00, 5.6000000e+01, 2.0000000e+01, 2.0212022e+07,
       1.0000000e+00, 0.0000000e+00, 3.3333300e-01, 1.0000000e+00,
       0.0000000e+00, 1.0000000e+00, 2.0000000e+00, 8.8889000e-02,
       0.0000000e+00, 0.0000000e+00, 8.8000000e+01])
```

[56]:
```
X = imputer.transform(numeric_hockey)
```

[57]:
```
X
```

[57]:
```
array([[  2.,  81.,  23., …,   0.,   0., 137.],
       [  2.,  79.,  38., …,   0.,   3., 194.],
       [  2.,   1.,   0., …,   0.,   0.,   2.],
       …,
       [  2.,  64.,  61., …,   0.,   0.,  99.],
       [  2.,   7.,   7., …,   0.,   0.,   1.],
       [  2.,  39.,   8., …,   0.,   0.,  55.]], shape=(4865, 15))
```

[58]:
```
hockey_tr = pd.DataFrame(X, columns=numeric_hockey.columns,
                         index=hockey.index)
```

[59]:
```
hockey_tr
```

[59]:

	gameTypeId	gamesPlayed	pim	season	sequence	plusMinus	\
3542	2.0	81.0	23.0	20242025.0	1.0	3.0	
1340	2.0	79.0	38.0	20212022.0	1.0	14.0	
5003	2.0	1.0	0.0	20152016.0	1.0	1.0	
5479	2.0	79.0	66.0	20212022.0	1.0	-3.0	
1960	2.0	29.0	0.0	20242025.0	2.0	6.0	
…	…	…	…	…	…	…	

```
4272         2.0         28.0   8.0  20192020.0       1.0        3.0
5130         2.0         30.0   8.0  20252026.0       1.0        1.0
5039         2.0         64.0  61.0  20212022.0       1.0      -13.0
3195         2.0          7.0   7.0  20242025.0       3.0        0.0
766          2.0         39.0   8.0  20182019.0       1.0       -8.0

      faceoffWinningPctg  gameWinningGoals  otGoals  powerPlayGoals  \
3542            0.454545               3.0      0.0             2.0
1340            0.535658               5.0      0.0             2.0
5003            0.142900               0.0      0.0             0.0
5479            0.000000               3.0      0.0             6.0
1960            0.250000               1.0      0.0             0.0
…                    …                 …        …               …
4272            0.481900               1.0      0.0             1.0
5130            0.542857               0.0      0.0             0.0
5039            0.564103               0.0      0.0             0.0
3195            0.000000               0.0      0.0             0.0
766             0.000000               0.0      0.0             1.0

      powerPlayPoints  shootingPctg  shorthandedGoals  shorthandedPoints  \
3542              5.0      0.167883               0.0                0.0
1340              4.0      0.139000               0.0                3.0
5003              0.0      0.500000               0.0                0.0
5479             15.0      0.069000               0.0                0.0
1960              1.0      0.115385               0.0                0.0
…                 …             …                  …                  …
4272              1.0      0.119000               0.0                0.0
5130              0.0      0.114286               0.0                0.0
5039              1.0      0.020000               0.0                0.0
3195              0.0      0.000000               0.0                0.0
766               1.0      0.054500               0.0                0.0

      shots
3542  137.0
1340  194.0
5003    2.0
5479  173.0
1960   52.0
…      …
4272   42.0
5130   35.0
5039   99.0
3195    1.0
766    55.0

[4865 rows x 15 columns]
```

1.4 Training

[60]:
```
from sklearn.linear_model import LinearRegression

hockey_prepared = hockey_tr
lin_reg = LinearRegression()
lin_reg.fit(hockey_prepared, hockey_labels)
```

[60]:
```
LinearRegression()
```

[61]:
```
some_data = hockey.iloc[:5]
some_labels = hockey_labels.iloc[:5]
some_data_prepared = some_data.select_dtypes(include='number')

print("Predictions:", lin_reg.predict(some_data_prepared))
```

```
Predictions: [37.37678905 47.35031953 13.28987096 46.45099008 13.82689966]
```

[62]:
```
print("Labels:", list(some_labels))
```

```
Labels: [35.0, 51.0, 1.0, 37.0, 14.0]
```

[63]:
```
some_data_prepared
```

[63]:

	gameTypeId	gamesPlayed	pim	season	sequence	plusMinus
3542	2.0	81.0	23.0	20242025.0	1.0	3.0
1340	2.0	79.0	38.0	20212022.0	1.0	14.0
5003	2.0	1.0	0.0	20152016.0	1.0	1.0
5479	2.0	79.0	66.0	20212022.0	1.0	-3.0
1960	2.0	29.0	0.0	20242025.0	2.0	6.0

	faceoffWinningPctg	gameWinningGoals	otGoals	powerPlayGoals
3542	0.454545	3.0	0.0	2.0
1340	0.535658	5.0	0.0	2.0
5003	0.142900	0.0	0.0	0.0
5479	0.000000	3.0	0.0	6.0
1960	0.250000	1.0	0.0	0.0

	powerPlayPoints	shootingPctg	shorthandedGoals	shorthandedPoints
3542	5.0	0.167883	0.0	0.0
1340	4.0	0.139000	0.0	3.0
5003	0.0	0.500000	0.0	0.0
5479	15.0	0.069000	0.0	0.0
1960	1.0	0.115385	0.0	0.0

	shots
3542	137.0
1340	194.0
5003	2.0

```
5479  173.0
1960   52.0
```

The model was not doing much, because it simply added goals and assists to always get the right amount of points but I removed assists assists so it could not "cheat".

[64]:
```
from sklearn.metrics import mean_squared_error

hockey_predictions = lin_reg.predict(hockey_prepared)
lin_mse = mean_squared_error(hockey_labels, hockey_predictions)
lin_rmse = np.sqrt(lin_mse)
lin_rmse
```

[64]:
```
np.float64(5.227128596183231)
```

[65]:
```
from sklearn.metrics import mean_absolute_error

lin_mae = mean_absolute_error(hockey_labels, hockey_predictions)
lin_mae
```

[65]:
```
3.856721847991256
```

[66]:
```
from sklearn.tree import DecisionTreeRegressor

tree_reg = DecisionTreeRegressor(random_state=42)
tree_reg.fit(hockey_prepared, hockey_labels)
```

[66]:
```
DecisionTreeRegressor(random_state=42)
```

[67]:
```
hockey_predictions = tree_reg.predict(hockey_prepared)
tree_mse = mean_squared_error(hockey_labels, hockey_predictions)
tree_rmse = np.sqrt(tree_mse)
tree_rmse
```

[67]:
```
np.float64(0.010137796748742194)
```

[68]:
```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(tree_reg, hockey_prepared, hockey_labels,
                         scoring="neg_mean_squared_error", cv=10)
tree_rmse_scores = np.sqrt(-scores)
```

[69]:
```
def display_scores(scores):
    print("Scores:", scores)
    print("Mean:", scores.mean())
    print("Standard deviation:", scores.std())

display_scores(tree_rmse_scores)
```

```
Scores: [7.36061443 7.43638317 7.82705779 8.27765107 7.75245871 7.62616786
 7.64866378 6.93710724 7.23958028 7.29986752]
Mean: 7.540555184185121
Standard deviation: 0.3528754087692715
```

[70]:
```
lin_scores = cross_val_score(lin_reg, hockey_prepared, hockey_labels,
                             scoring="neg_mean_squared_error", cv=10)
lin_rmse_scores = np.sqrt(-lin_scores)
display_scores(lin_rmse_scores)
```

```
Scores: [5.21904132 5.09161879 5.02525589 5.77866173 5.48405549 5.22528977
 4.93329534 5.20092322 5.31129616 5.19520023]
Mean: 5.246463794517647
Standard deviation: 0.22820480142858185
```

It is indeed better to use the regression so the decision tree is most likely overfitting...

[71]:
```
from sklearn.ensemble import RandomForestRegressor

forest_reg = RandomForestRegressor(n_estimators=100, random_state=42)
forest_reg.fit(hockey_prepared, hockey_labels)
```

[71]:
```
RandomForestRegressor(random_state=42)
```

[72]:
```
hockey_predictions = forest_reg.predict(hockey_prepared)
forest_mse = mean_squared_error(hockey_labels, hockey_predictions)
forest_rmse = np.sqrt(forest_mse)
forest_rmse
```

[72]:
```
np.float64(1.8716945522210557)
```

[73]:
```
from sklearn.model_selection import cross_val_score

forest_scores = cross_val_score(forest_reg, hockey_prepared, hockey_labels,
                                scoring="neg_mean_squared_error", cv=10)
forest_rmse_scores = np.sqrt(-forest_scores)
display_scores(forest_rmse_scores)
```

```
Scores: [5.1350253  5.13785848 4.80325185 5.98547377 5.05601319 5.1491064
 5.0597325  4.7590253  5.02342843 4.7733368 ]
Mean: 5.0882252028636215
Standard deviation: 0.3328964390047266
```

[74]:
```
# This is broken I think
scores = cross_val_score(lin_reg, hockey_prepared, hockey_labels,␣
 ↪scoring="neg_mean_squared_error", cv=10)
pd.Series(np.sqrt(-scores)).describe
```

[74]:
```
<bound method NDFrame.describe of 0    5.219041
1    5.091619
2    5.025256
3    5.778662
4    5.484055
5    5.225290
6    4.933295
7    5.200923
8    5.311296
9    5.195200
dtype: float64>
```

In my case the regression seems to perform better

[77]:
```
# Coefficients (weights)
coefficients = lin_reg.coef_


# Displaying coefficients with feature names
coef_df = pd.DataFrame({
    'Feature': hockey_prepared.columns,
    'Coefficient': coefficients
})
print(coef_df)
```

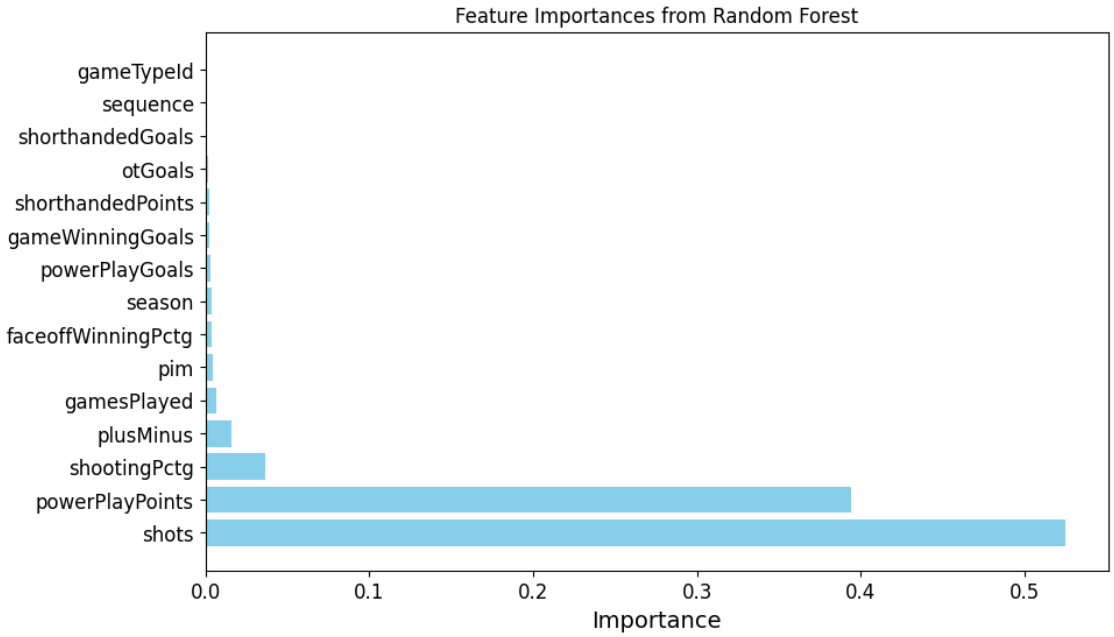
```
              Feature  Coefficient
0          gameTypeId     0.000000
1         gamesPlayed     0.113504
2                 pim    -0.014671
3              season     0.000018
4            sequence    -0.576019
5           plusMinus     0.202399
6   faceoffWinningPctg     3.068115
7     gameWinningGoals     0.717495
8              otGoals     0.579285
9       powerPlayGoals    -0.400300
10     powerPlayPoints     1.397000
11        shootingPctg    33.074000
12    shorthandedGoals    -0.088174
13   shorthandedPoints     1.030053
14               shots     0.110941
```

[76]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Get feature importances
feature_importances = forest_reg.feature_importances_
```

```
# Creating a DataFrame for better visualization
importance_df = pd.DataFrame({
    'Feature': hockey_prepared.columns,
    'Importance': feature_importances
}).sort_values(by='Importance', ascending=False)

# Plotting the feature importances
plt.figure(figsize=(10, 6))
plt.barh(importance_df['Feature'], importance_df['Importance'], color='skyblue')
plt.xlabel('Importance')
plt.title('Feature Importances from Random Forest')
plt.show()
```



So if you want your players to make points, according to this model, they have to shoot the puck and obviously make points on the powerplay. I see why this logic is flawed, because powerplay points are points, but you could arue they represent time spent on the powerplay, which is not available witht this dataset.

Obviously shot percentage and shots give a close estimate of the goals, so the model "gets" goals with shots and shot percentage and gets exaggerated points with the power play points which gives a close estimate.